

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

TP1 — Base architecturale propre

Objectifs pédagogiques

Construire un monolithe propre et pédagogique servant de socle aux TP suivants.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

Jour 1 et Jour 2

Table des matières

- **Objectifs**
- Énoncé
- Étapes
- Structure
- Code
- Pièges
- Évaluation
- Corrigé

Objectifs pédagogiques

- Structurer une application Spring Boot propre.
- Séparer contrôleurs, services, domaine, persistance.
- Utiliser DTO, validation, gestion d'erreurs, documentation.
- Préparer un futur passage au distribué.

Énoncé

Créer une application **Campus Library Flow** monolithique permettant :

- gérer des livres ;
- créer un prêt ;
- retourner un prêt ;
- consulter les prêts d'un étudiant.

Prérequis

Java 21, Spring Boot, Spring Web, Spring Data JPA, validation Bean Validation, base H2 ou PostgreSQL.

Étapes détaillées

1. Créer le projet Spring Boot.
2. Définir les packages : `web`, `application`, `domain`, `infrastructure`.
3. Créer les entités métier `Book` et `Loan`.
4. Créer les DTO d'entrée/sortie.
5. Implémenter la validation.
6. Ajouter une gestion d'erreurs centralisée.
7. Documenter l'API avec OpenAPI / Swagger.

Structure de projet

```
src/main/java/com/example/library
├── web
│   ├── BookController.java
│   ├── LoanController.java
│   └── ApiExceptionHandler.java
├── application
│   ├── BookApplicationService.java
│   └── LoanApplicationService.java
├── domain
│   ├── Book.java
│   ├── Loan.java
│   └── BusinessException.java
└── infrastructure
    ├── persistence
    └── mapper
```

Extraits de code commentés

```
// DTO de création de Livre
public record CreateBookRequest(
    @NotBlank String title,
    @NotBlank String author,
    @Positive int stock
) {}
```

```
// Gestion d'erreurs centralisée : évite les try/catch partout
@RestControllerAdvice
public class ApiExceptionHandler {}

@ExceptionHandler(BusinessException.class)
public ResponseEntity<Map<String, String>> handleBusiness(BusinessException ex) {{
    return ResponseEntity.badRequest()
        .body(Map.of("error", ex.getMessage()));
}}
}}
```

Pièges fréquents

- Mettre la logique métier dans le contrôleur.
- Retourner directement l'entité JPA.
- Écrire les règles métier dans le repository.
- Ne pas distinguer erreur technique et erreur métier.

Critères d'évaluation

Critère	Points d'attention
Structure	Séparation claire des couches
Qualité API	DTO, codes HTTP, validation
Gestion d'erreurs	Centralisée et compréhensible
Clarté du code	Nommage, simplicité, commentaires utiles
Préparation au futur	Couplage limité, structure extensible

Corrigé détaillé

Le corrigé complet attendu :

- contrôleurs fins ;
- services d'application qui orchestrent ;
- modèle métier capable d'exprimer les règles de retour et de disponibilité ;
- mapping explicite ;
- OpenAPI ;
- tests basiques de service et de contrôleur.

À la fin, l'étudiant doit avoir compris

Qu'avant de parler microservices, il faut maîtriser la structuration interne d'un système unique.

[← Jour 6](#)

[TP2 →](#)